

The Hopfield Network: A Nostalgic Mind in the Neural Wilderness

Exploring the dynamic memory landscapes of one of the most enigmatic recurrent architectures.

Hernández Pérez Erick Fernando

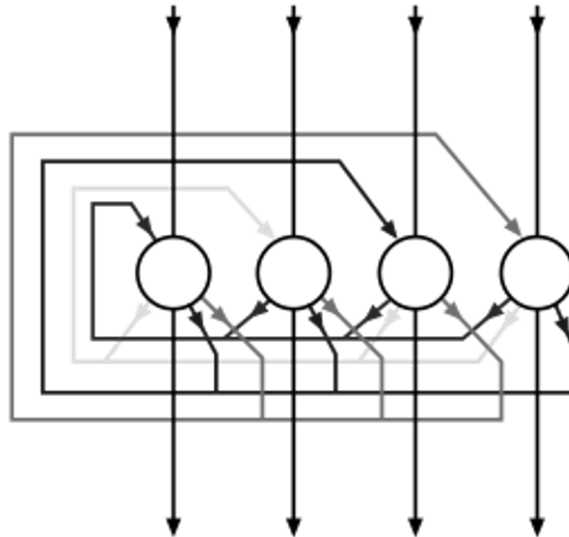


Figure 1

One could easily write a book of species on the various models of neural networks—so vast a taxonomy that it would undoubtedly be worthy of Carl Linnaeus himself. From simple models like the McCulloch-Pitts neurons or Rosenblatt’s perceptron, to grander specimens such as convolutional neural networks—with architectures as imposing as LeNet, AlexNet, or ResNet.

The ecosystem is undoubtedly broad, but it is time to turn our attention to a curious specimen that often eludes the common body of knowledge; not due to a lack of application or potential, but rather because of a subtle veil of unfamiliarity that still hovers over the place it inhabits. Its name is the Hopfield network.

This type of network belongs to the family of recurrent networks—that is, the outputs of the neurons also serve as inputs for the next computation. Its behavior is dynamic: it evolves over time until reaching a stable state, as if the system were slowly settling into a resting position within an energy landscape, where each valley represents a learned configuration.

Unlike other models that follow straightforward paths to an output, Hopfield networks move in spirals, as if rummaging through hazy memories to find the nearest matching pattern. This quality makes them particularly useful for recognizing and reconstructing incomplete or corrupted patterns—much like a blurry image coming into focus from just a few well-placed clues. When presented with a partial input, the network remembers—in a literal sense of associative memory—the closest complete pattern it previously learned.

Thus, rather than predicting, Hopfield networks remember. And they do so with an elegance that only well-crafted systems possess: they converge, stabilize, and retrieve what was once taught to them, without the need for ongoing supervision. They are, in a sense, a kind of nostalgic memory within the vast kingdom of neural networks.

The Notation

We follow the notation given for Mehlig [1]. We consider p patterns indexed by $\mu = 1, \dots, p$. The bits of pattern μ are denoted by $x_i^{(\mu)}$. The index $i = 1, \dots, N$ labels the different bits of a given pattern, and N is the number of bits per pattern.

The bits are binary, they only can take the values -1 and +1. Its convenient to gather the bits of a pattern in a column vector:

$$\mathbf{x}^{(\mu)} = \begin{bmatrix} x_1^{(\mu)} \\ x_2^{(\mu)} \\ \vdots \\ x_N^{(\mu)} \end{bmatrix}$$

“The task of the neural network is to recognize distorted patterns” (Mehlig, 2021). We could say that this process resembles a form of memorization and recall: when presented with a given pattern, the network identifies the most similar one it has previously learned.

To make things more concrete, We store p patterns in the neural network and then present it with a distorted version of one of them. The network is then asked to retrieve the stored pattern that best matches the distorted input.

But, how we can define how similar a distorted pattern $\mathbf{x}$ is to any of the stored patterns $\mathbf{x}^{(\mu)}$?. We define the distance d_μ between the patterns $\mathbf{x}$ and $\mathbf{x}^{(\mu)}$ in terms of the mean squared error:

$$d_\mu = \frac{1}{4N} \sum_{i=1}^N (x_i - x_i^{(\mu)})^2$$

We use $\frac{1}{4N}$ is chosen so that the distance equals the fraction of bits by which two ± 1 patterns differ. So, the distance tells us how many bits differ between each pattern.

The Hebb's Rule

The Hopfield networks are networks of McCulloch-Pitts neurons (see [3. Implementing McCulloch and Pitts Neuron in Python](#)) designed to solve the pattern-recognition problem.

The bits of the patterns are encoded through a specific choice of weights known as Hebb's rule. But before addressing that, we need to analyze the function of the network itself.

The possible state of the McCulloch-Pitts neurons in the network are

$$\mathbf{s} = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_N \end{bmatrix}$$

$\mathbf{s}$ forms the configuration space of the network. The components of the state vector $\mathbf{s}$ are updated according to the synchronous rule:

$$b_i(t) = \sum_{j=1}^N w_{ij} s_j(t) - \theta_i$$

$$s_i(t+1) = \text{sgn}(b_i(t))$$

The first step in recognizing a distorted pattern is to feed its bits x_i into the network. Thus, we set:

$$s_i(t=0) = x_i$$

Now, we need to choose a set of weights w_{ij} such that the network converges to the correct stored pattern. For instance, if $\mathbf{x}$ is a distorted version of $\mathbf{x}^{(\mu)}$, then we want:

$$\mathbf{s}(t=0) = \mathbf{x} \approx \mathbf{x}^{(\mu)} \implies \mathbf{s}(t) \rightarrow \mathbf{x}^{(\mu)} \quad \text{as } t \rightarrow \infty$$

Let us begin at the beginning: how does the network detect a pattern $\mathbf{x}^{(\mu)}$ as one of the stored ones? More formally, we require:

$$\mathbf{s}(t=0) = \mathbf{x}^{(\mu)} \implies \mathbf{s}(t) = \mathbf{x}^{(\mu)} \quad \forall \quad t = 0, 1, 2, \dots$$

Let us consider a simple case first, where is only one pattern $\mathbf{x}^{(1)}$. We can choose the following weights and thresholds:

$$w_{ij} = \frac{1}{N} x_i^{(1)} x_j^{(1)} \\ \theta_i = 0$$

This learning rule reminds of a relation between activation patterns of neurons and their coupling, postulated by Hebb more than 70 years ago (Mehlig, 2021):

When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased.

This learning process strengthens connections: the coupling between neurons increases if they are active at the same time. Let us now see whether the Hopfield network can recognize the pattern $\mathbf{x}^{(1)}$:

$$\sum_{j=1}^N w_{ij} x_j^{(1)} = \frac{1}{N} \sum_{j=1}^N x_i^{(1)} x_j^{(1)} x_j^{(1)} = \frac{1}{N} \sum_{j=1}^N x_i^{(1)} = x_i^{(1)}$$

Finally, applying the sign function:

$$\text{sgn} \left(\sum_{j=1}^N w_{ij} x_j^{(1)} \right) = x_i^{(1)}$$

Therefore, the pattern remains unchanged, and the network correctly recognizes it as one of the stored patterns.

But does the network actually correct errors? Is the pattern $\mathbf{x}^{(1)}$ an attractor? This question cannot be answered in general. Nevertheless, in practice, Hopfield networks often perform remarkably well.

It is a fundamental insight that neural networks can yield good results even when there is no formal proof that their dynamics will converge to the correct solution.

For now, returning to our main discussion, the general form of the weight matrix for a single pattern of N bits is given by:

$$W = \frac{1}{N} \mathbf{x}^{(1)} \mathbf{x}^{(1)T}$$

More than one pattern

We now generalize Hebb's rule to p patterns:

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^p x_i^{(\mu)} x_j^{(\mu)} \\ \theta_i = 0$$

An alternative version of Hebb's rule (and more common) sets the diagonal weights to zero:

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^p x_i^{(\mu)} x_j^{(\mu)} \quad \text{for} \quad i \neq j \\ w_{ii} = 0 \\ \theta_i = 0$$

Now we need to check whether this form recognizes the stored patterns. In other words, we need to verify whether the following holds:

$$\text{sgn} \left(\frac{1}{N} \sum_{i \neq j} \sum_{\mu} x_i^{(\mu)} x_j^{(\mu)} x_j^{(\nu)} \right) = x_i^{(\nu)}$$

To determine whether a stored pattern is a fixed point of the Hopfield network dynamics, we examine the local field acting on neuron i when the network is presented with one of the stored patterns, say $\mathbf{x}^{(1)}$. The local field is defined as:

$$b_i = \sum_{j=1}^N W_{ij} x_j^{(1)},$$

where W_{ij} is the weight matrix, constructed using the Hebbian learning rule for p stored patterns:

$$W_{ij} = \frac{1}{N} \sum_{\mu=1}^p x_i^{(\mu)} x_j^{(\mu)} \quad \text{with} \quad W_{ii} = 0.$$

Substituting into the local field expression, we obtain:

$$\begin{aligned} b_i &= \sum_{j=1}^N \left(\frac{1}{N} \sum_{\mu=1}^p x_i^{(\mu)} x_j^{(\mu)} \right) x_j^{(1)} \\ &= \frac{1}{N} \sum_{\mu=1}^p x_i^{(\mu)} \sum_{j=1}^N x_j^{(\mu)} x_j^{(1)}. \end{aligned}$$

We now separate the contribution from the pattern $\mu = 1$, which is the one we wish the network to recognize, from the rest:

$$b_i = \frac{1}{N} x_i^{(1)} \sum_{j=1}^N x_j^{(1)} x_j^{(1)} + \frac{1}{N} \sum_{\mu=2}^p x_i^{(\mu)} \sum_{j=1}^N x_j^{(\mu)} x_j^{(1)}.$$

Since $x_j^{(1)} \in \{-1, +1\}$, we have $x_j^{(1)} x_j^{(1)} = 1$, so the first term simplifies to:

$$\frac{1}{N} x_i^{(1)} \sum_{j=1}^N 1 = x_i^{(1)}.$$

However, because $W_{ii} = 0$, the self-connection must be excluded from the sum. Therefore, the corrected local field becomes:

$$b_i = x_i^{(1)} \left(1 - \frac{1}{N} \right) + \text{cross-talk term},$$

where the cross-talk term contains the interference from the other stored patterns ($\mu \neq 1$). For the network to recognize the pattern $\mathbf{x}^{(1)}$ correctly, this interference must not alter the sign of b_i .

Whether adding the cross-talk term to $x_i^{(1)}$ affects the sign of the local field b_i depends on the specific set of stored patterns. The cross-talk term contains a sum over all stored patterns $\mu = 1, \dots, p$, and its contribution becomes more significant as p increases.

However, when the number of stored patterns p is small relative to the number of neurons N , the cross-talk term tends to be small. In such cases, it is unlikely to change the sign of b_i , and the network is more likely to correctly recognize the stored patterns.

It is plausible that the stored patterns are not only fixed points but also attractors of the network dynamics. This means that slightly distorted versions of the stored patterns are expected to converge to the correct pattern over time.

Limitations of Hopfield Networks

While Hopfield networks offer an elegant framework for associative memory, they are not without limitations. Understanding these drawbacks is crucial for applying the model effectively and for appreciating its extensions in modern neural network theory.

1. Limited Storage Capacity

The number of patterns that can be reliably stored and recalled in a Hopfield network is limited. For a network with N neurons, the practical storage capacity is approximately:

$$p_{\max} \approx 0.138N,$$

beyond which the probability of successful recall sharply decreases due to interference (cross-talk) between stored patterns.

2. Spurious States

In addition to the stored patterns, Hopfield networks can converge to unintended stable states, known as *spurious states*. These can include:

- Linear combinations (or mixtures) of stored patterns,
- Patterns that are not part of the training set but emerge due to network dynamics,
- Inverted patterns (negatives of the originals) if not explicitly controlled.

Spurious states reduce the reliability of the network and may lead to incorrect or ambiguous outputs.

3. Symmetric Weight Matrix Requirement

Hopfield networks require the weight matrix W to be symmetric ($W_{ij} = W_{ji}$) and have no self-connections ($W_{ii} = 0$). While this ensures convergence to stable states, it also limits the network's expressiveness and adaptability.

4. Binary Neuron Outputs

The original Hopfield model uses binary neurons (± 1), which restricts the kind of information it can store and recall. Continuous or more complex data representations require modifications or generalizations of the model.

5. Inefficiency for Large-Scale Problems

Hopfield networks are not well suited for high-dimensional or large-scale problems. Their computational cost and tendency to converge to local minima make them inefficient compared to modern deep learning architectures.

Despite these limitations, Hopfield networks remain a foundational model in neural computation, offering deep insights into memory, energy-based learning, and dynamical systems.

References

- [1] Mehlig, B. (2021). *Machine learning with neural networks*. Department of Physics, University of Gothenburg.